

Graphics in R for Biomedicine

David Moreno

Introduction and philosophy

Why ggplot2?

ggplot2 implements the **Grammar of Graphics** (Wilkinson 2005), a system where each plot is built by combining independent layers:

Layer	Function	Example
Data	data.frame (database)	<code>ggplot(my_data, ...)</code>
Aesthetics	Visual attributes	<code>aes(x, y, color, size)</code>
Geometries	Type of visual mark	<code>geom_point(),</code> <code>geom_boxplot()</code>
Scales	How to map values to visual space	<code>scale_color_manual()</code>
Facets	Sub-plots by categorical variable	<code>facet_wrap(~group)</code>
Theme	Overall appearance	<code>theme_classic()</code>

Most scientific figures can be created using ggplot. However, some plots are complex and requires a lot of coding and/or parameters. In the latter cases, I recommend to use specific packages depending on the context (heatmaps, volcano plots, kaplan-meier curves, ...).

Then, what is the relevance of knowing the grammar of graphics? Most specific packages for plotting rely on the same layers (data/aesthetics/geometries, etc). It allows you to improve the figure as you want.

Data we will use

Use the myeloma data available in the survminer package:

```
# Install packages if needed:
# install.packages(c("ggplot2", "ggpubr", "survival", "survminer",
#                   "pheatmap", "dplyr", "tidyr", "ggrepel"))

library(ggplot2)
library(ggpubr)
library(survival)
library(survminer)
```

Attaching package: 'survminer'

The following object is masked from 'package:survival':

myeloma

```
library(pheatmap)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(tidyr)
library(ggrepel)

data("myeloma")
myeloma

head(myeloma, 4)
```

Scatter plots

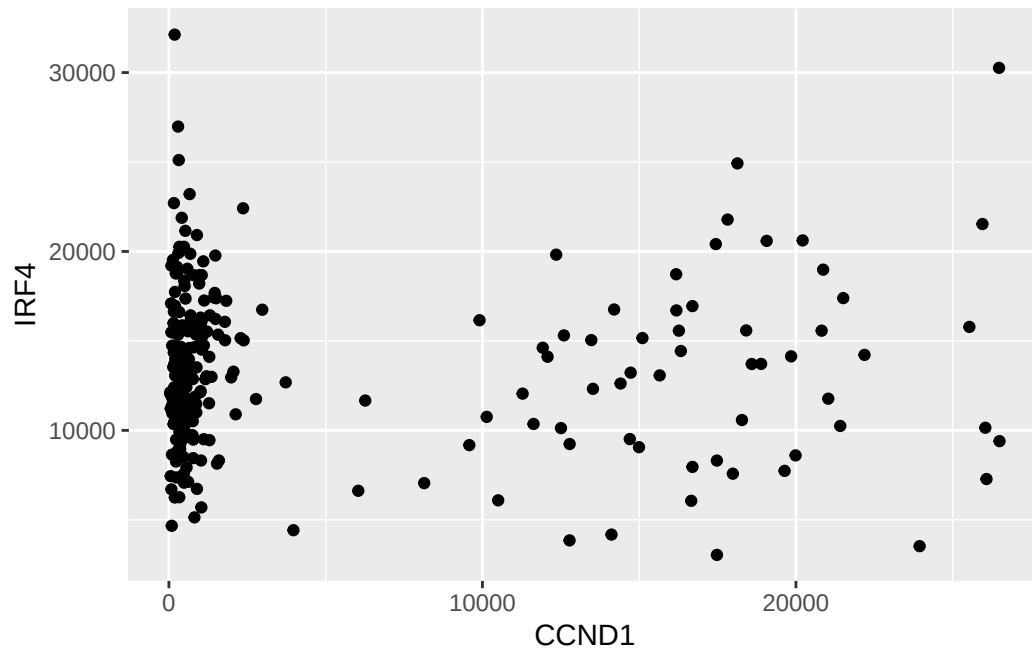
Basic scatter plot

To explore the relationship between two continuous variables.

```
data("myeloma")
head(myeloma)
```

	molecular_group	chr1q21_status	treatment	event	time	CCND1	CRIM1
GSM50986	Cyclin D-1	3 copies	TT2	0	69.24	9908.4	420.9
GSM50988	Cyclin D-2	2 copies	TT2	0	66.43	16698.8	52.0
GSM50989	MMSET	2 copies	TT2	0	66.50	294.5	617.9
GSM50990	MMSET	3 copies	TT2	1	42.67	241.9	11.9
GSM50991	MAF	<NA>	TT2	0	65.00	472.6	38.8
GSM50992	Hyperdiploid	2 copies	TT2	0	65.20	664.1	16.9
	DEPDC1	IRF4	TP53	WHSC1			
GSM50986	523.5	16156.5	10.0	261.9			
GSM50988	21.1	16946.2	1056.9	363.8			
GSM50989	192.9	8903.9	1762.8	10042.9			
GSM50990	184.7	11894.7	946.8	4931.0			
GSM50991	212.0	7563.1	361.4	165.0			
GSM50992	341.6	16023.4	2096.3	569.2			

```
# Data, aesthetics, geometries
ggplot(data = myeloma, # data
       aes(x = CCND1, y = IRF4)) + # aesthetics/variables
  geom_point() # geometries
```



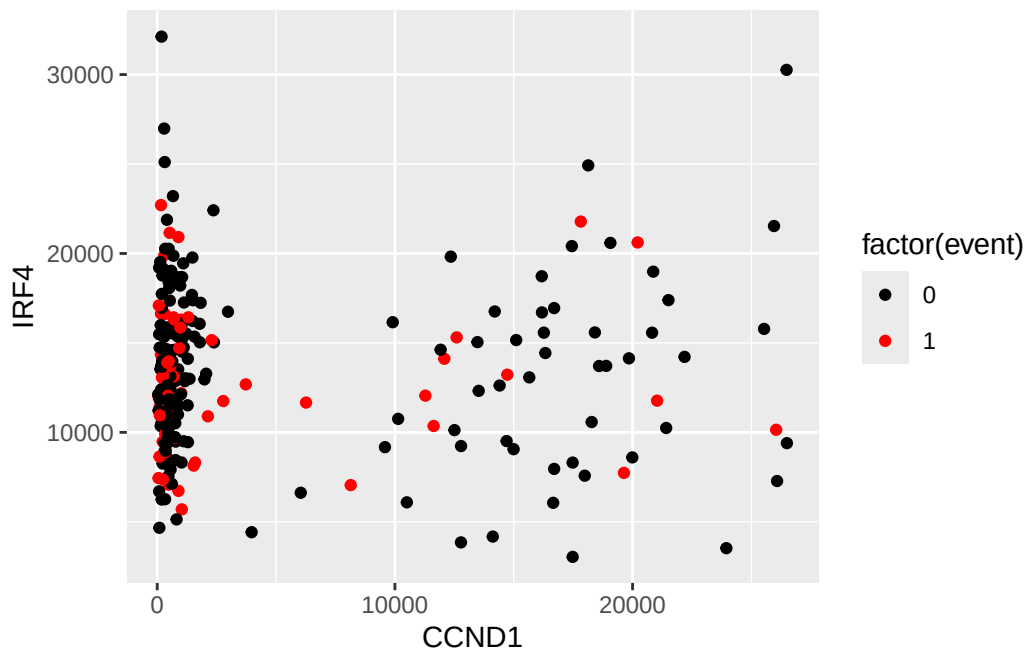
```
table(myeloma$event)
```

```
0  1
186 70
```

```
class(myeloma$event)
```

```
[1] "integer"
```

```
ggplot(data = myeloma, # data
       aes(x = CCND1, y = IRF4, color = factor(event))) + # aesthetics/variables
geom_point() + # geometries
scale_color_manual(values = c("1" = "red", "0" = "black")) # scales
```



Adding linear regression and correlation

`geom_smooth(method = "lm")` draws a regression line with 95% confidence interval. The `stat_cor()` function from `ggpubr` adds the Pearson correlation coefficient (r) and its p-value directly on the plot.

Statistics: Pearson correlation

The coefficient r measures the strength and direction of the linear association between two continuous variables. It ranges from -1 (perfect negative) to $+1$ (perfect positive). A value of $r = 0$ indicates no linear relationship.

```
# Use the flchain dataset from the survival package
```

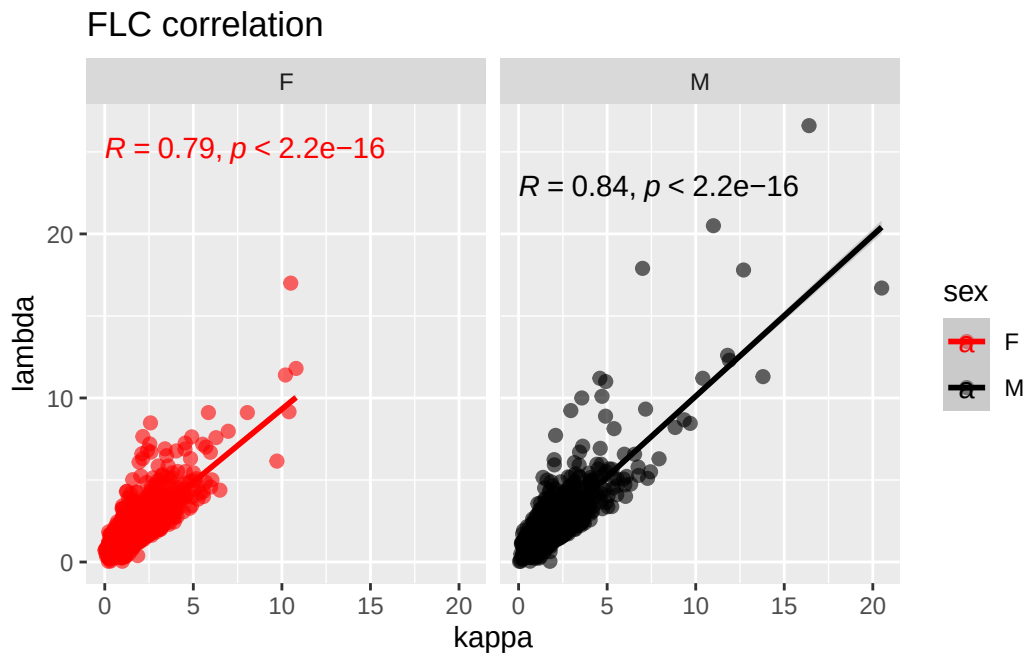
```
data("flchain")
head(flchain)
```

	age	sex	sample.yr	kappa	lambda	flc.grp	creatinine	mgus	futime	death
1	97	F	1997	5.70	4.860	10	1.7	0	85	1
2	92	F	2000	0.87	0.683	1	0.9	0	1281	1
3	94	F	1997	4.36	3.850	10	1.4	0	69	1
4	92	F	1996	2.42	2.220	9	1.0	0	115	1
5	93	F	1996	1.32	1.690	6	1.1	0	1039	1

6	90	F	1997	2.01	1.860	9	1.0	0	1355	1
chapter										
1	Circulatory									
2	Neoplasms									
3	Circulatory									
4	Circulatory									
5	Circulatory									
6	Mental									

```
ggplot(flchain, aes(x = kappa, y = lambda, color = sex)) +
  geom_point(size = 2, alpha = 0.6) +
  geom_smooth(method = "lm", se = TRUE) +
  stat_cor() + # try using options of stat_cor
  scale_color_manual(values = c("M" = "black", "F" = "red")) +
  facet_wrap(~sex) + # try using scales
  labs(title = "FLC correlation",
       x = "kappa", y = "lambda")
```

`geom\_smooth()` using formula = 'y ~ x'



Comparing groups: boxplots and violins

Boxplot: anatomy and interpretation

A boxplot summarises the distribution of a continuous variable. Before interpreting differences between groups, it is essential to understand what each element represents.

Statistics: tests for comparing groups

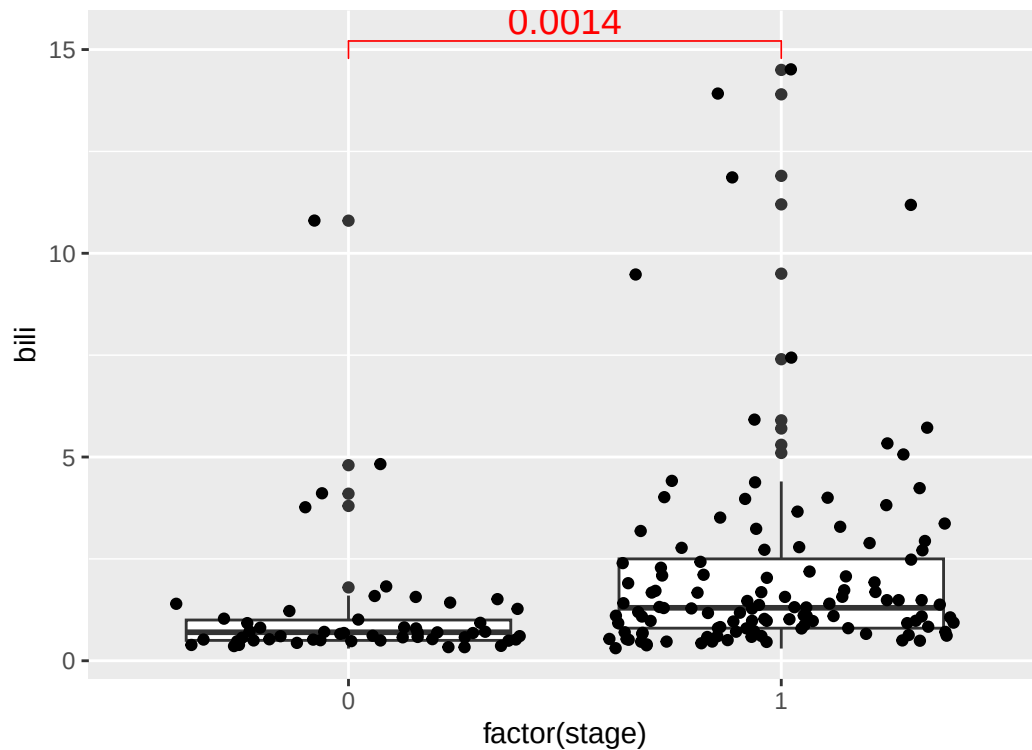
Table 2: **Non-parametric tests** (Wilcoxon, Kruskal–Wallis) do not assume normality and are preferable when $n < 30$ per group, data have severe outliers, or the variable is an ordinal scale. **Parametric tests** are more powerful when their assumptions hold (normality, homogeneity of variances).

Situation	Parametric test	Non-parametric test
2 groups	Student's t	Wilcoxon–Mann–Whitney
3 groups	ANOVA	Kruskal–Wallis

```
data("udca")
head(udca1)
```

```
   id trt stage bili riskscore futime status
1  1  1    1    1  1.0         5.1   1896     0
2  2  2    0    1  1.7         4.2   1456     1
3  3  3    1    0  0.5         3.4   1892     0
4  4  4    1    1  1.4         5.0    343     0
5  5  5    0    1  1.1         4.3   1875     0
6  6  6    1    1  1.4         5.9    768     1
```

```
# Compare bilirubin distribution across stage
# Build the boxplot from 0 and explore options using stat_compare_means
ggplot(udca1, aes(x= factor(stage), y = bili)) +
  geom_boxplot() +
  geom_jitter() +
  stat_compare_means(method = "t.test", label.x.npc = "center", color = "red", size = 5, comp
```



```
# Rename the stage (0 = "low", 1 = "high")
# Change the color of stage low and stage high
# Add a title
# Change the axis labels
# How to improve the plot?
```

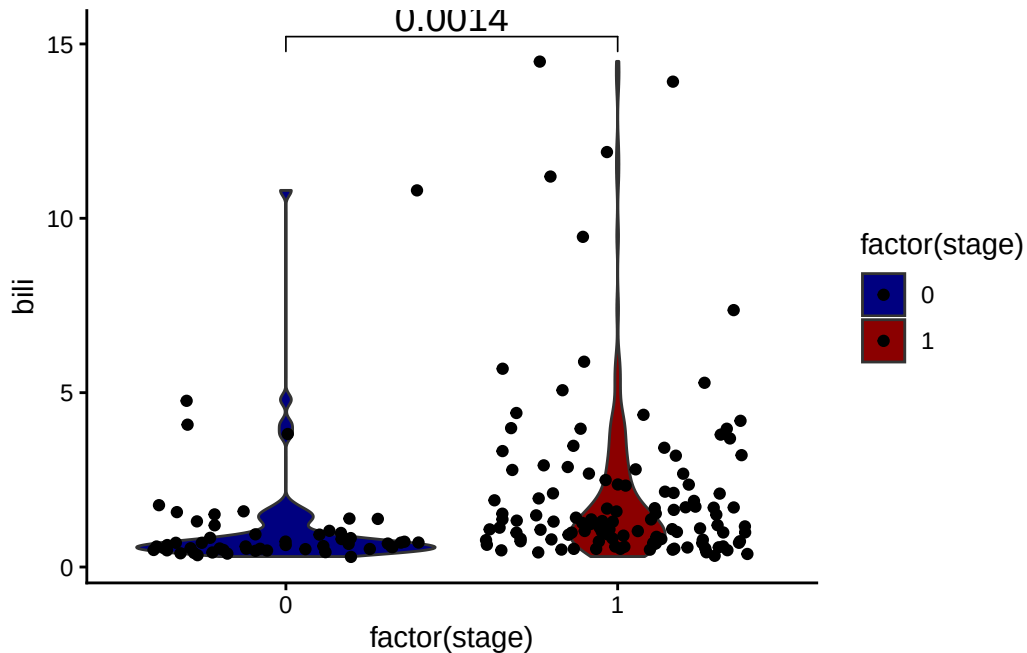
Violin plot

The violin plot combines the information of the boxplot with the probability density, showing the actual shape of the distribution.

```
ggplot(udca1, aes(x= factor(stage), y = bili, fill = factor(stage))) +
  geom_violin() +
  geom_jitter() +
  stat_compare_means(method = "t.test", label.x.npc = "center", color = "black", size = 5,

  scale_fill_manual(values = c("0" = "navy", "1" = "darkred")) +

  theme_classic()
```

When to use violin vs boxplot:

- **Boxplot:** symmetric distributions, moderate n, need to see outliers.
- **Violin:** bimodal or highly skewed distributions, large n, want to see the full shape.

Kaplan–Meier survival curves

Key concepts

Survival analysis models the **time to an event** (death, relapse, hospitalisation...). Its key features:

- **Censored:** patients who have not experienced the event by end of follow-up.
- **Survival function $S(t)$:** probability of surviving beyond time t .
- **Kaplan–Meier estimator:** estimates $S(t)$ non-parametrically, updating the probability at each event.

Log-rank test: compares curves between groups. Tests H_0 : “the survival curves are equal”. Assumes no parametric distribution.

Hazard Ratio (HR): in the Cox model, $HR > 1$ indicates higher relative risk in the exposure group; $HR < 1$ indicates a protective effect.

```
# Use the "lung" dataset from the survival package
library(survival)
lung <- survival::lung

fit <- survfit(Surv(time, status==2) ~ sex, data = lung) # status=1 alive, status=2 dead
#head(summary(fit))

# ggsurvplot is in the package survminer
# ggsurvfit is in the package ggsurvfit

library(ggsurvfit)
library(survminer)

#ggsurvplot(fit) # Improve the plot
#ggsurvfit(fit) # Improve the plot, use scale_color_manual, geom_vline, geom_hline, annotate
```

Gene expression heatmaps

What does a heatmap show?

A heatmap represents a **numerical matrix** using a colour gradient. In genomics, the matrix is typically genes per samples with expression values (counts, FPKM, log2FC...).

Statistics: z-score normalisation

Before visualising, it is common to standardise the expression of each gene (row) with the z-score:

$$z_{ij} = \frac{x_{ij} - \bar{x}_i}{\sigma_i}$$

This centres each gene at 0 and scales it to unit variance, allowing comparison of genes with very different expression ranges. In **heatmap**, this is activated with `scale = "row"`.

Hierarchical clustering (dendrogram) groups similar genes and samples using Euclidean distances and Ward's method (minimises within-cluster variance).

```
# Let's use ComplexHeatmap or pheatmap
# BiocManager::install("ComplexHeatmap")
# install.packages("pheatmap")

library(pheatmap)
library(ComplexHeatmap, verbose=F)
```

Loading required package: grid

```
=====
ComplexHeatmap version 2.22.0
Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
Github page: https://github.com/jokergoo/ComplexHeatmap
Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
```

If you use it in published research, please cite either one:

- Gu, Z. Complex Heatmap Visualization. iMeta 2022.
- Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional genomic data. Bioinformatics 2016.

The new InteractiveComplexHeatmap package can directly export static complex heatmaps into an interactive Shiny app with zero effort. Have a try!

This message can be suppressed by:

```
suppressPackageStartupMessages(library(ComplexHeatmap))
```

```
=====
! pheatmap() has been masked by ComplexHeatmap::pheatmap(). Most of the arguments
  in the original pheatmap() are identically supported in the new function. You
  can still use the original function by explicitly calling pheatmap::pheatmap().
```

Attaching package: 'ComplexHeatmap'

The following object is masked from 'package:pheatmap':

```
pheatmap
```

```

# Create a matrix from scratch
#rnorm(100)

mat_1 <- matrix(rnorm(100, mean = 20), ncol = 10) # Let's say this is "Myeloma"
mat_2 <- matrix(rnorm(100, mean = 10), ncol = 10) # Let's say this is "MGUS"
#mat_global <- cbind...

#pheatmap(mat_global) # is this correct?
#Heatmap(mat_global) # is this correct?

# Try to bind both mat_1 and mat_2 and create a heatmap. Give names to matrix samples
# Scale data
# Try options: give names to rows

#Heatmap(t(new_scale), row_names_side = NULL) +
# rowAnnotation(label = anno_mark(at = which(rownames(t(new_scale)) %in% genes_to_mark), #1

```

Volcano plot — differential expression

Interpretation

The **volcano plot** simultaneously represents:

- **X axis:** log Fold Change (magnitude of change). Positive = overexpressed in case vs control.
- **Y axis:** $-\log$ (adjusted p-value). Higher = more significant.

Genes of interest are in the upper corners (large change AND significant). Those at the base have a significant but small change (little biological relevance) or vice versa.

Statistics: multiple testing correction in genomics

In an RNA-seq experiment with 10,000 to 20,000 genes, using $\alpha = 0.05$ without correction would yield ~1,000 false positives by chance.

The **Benjamini–Hochberg FDR correction** controls the expected proportion of false positives among significant findings. In practice, **FDR < 0.05 and $|\log \text{FC}| > 1$** is used as the standard threshold. Adjusted p-values are commonly called **q-values** or **padj**.

```

# Use the all_markers_tutorial in the github link
# Install the EnhancedVolcano package
#BiocManager::install("EnhancedVolcano")

library(EnhancedVolcano)
dge_genes <- read.delim("/Users/dfmoreno/Downloads/all_markers_tutorial (2).txt", sep="")

# Explore the dge_genes dataset
# Run the EnhancedVolcano with default parameters
EnhancedVolcano(dge_genes,
                 lab = dge_genes$gene,
                 x = "avg_log2FC",
                 y = "p_val")

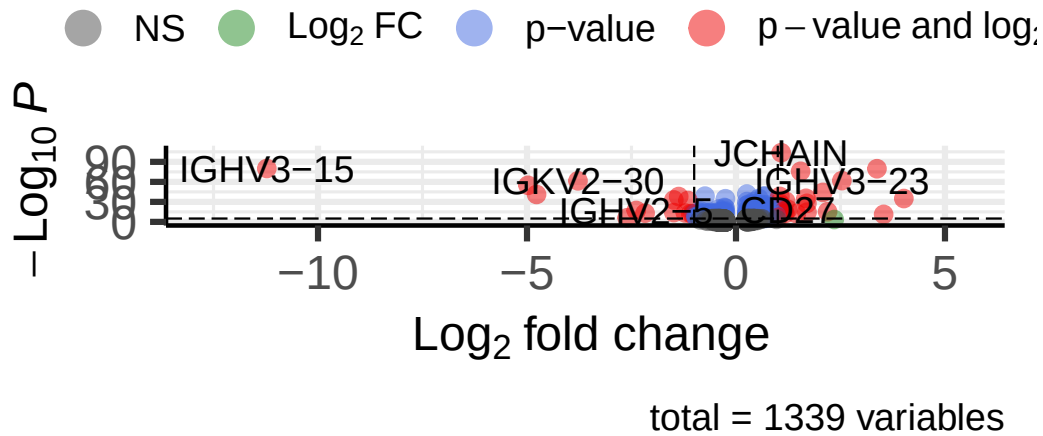
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.
 i The deprecated feature was likely used in the EnhancedVolcano package.
 Please report the issue to the authors.

Warning: The `size` argument of `element\_line()` is deprecated as of ggplot2 3.4.0.
 i Please use the `linewidth` argument instead.
 i The deprecated feature was likely used in the EnhancedVolcano package.
 Please report the issue to the authors.

Volcano plot

EnhancedVolcano



Improve the plot: How would you improve this plot?

Exporting figures for publication

Most scientific journals require figures at **minimum 300 DPI** in TIFF, EPS, or PDF format.

```
# Use ggsave
# Use pdf

# ggsave as pdf
ggsave(filename = "figure1_volcano.pdf", width = 8, height = 6, units = "cm", dpi = 300)

# pdf
pdf("/Users/dmoreno/Downloads/heatmap_expression.pdf", width = 10, height = 8)
EnhancedVolcano(dge_genes,
  lab = dge_genes$gene,
  x = "avg_log2FC",
  y = "p_val")
```

```
dev.off()
```

```
# Further editing: consider using an image editor
```

```
sessionInfo()
```

```
R version 4.4.3 (2025-02-28)
```

```
Platform: aarch64-apple-darwin20
```

```
Running under: macOS 26.5
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Madrid
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] grid      stats      graphics  grDevices  utils      datasets  methods
```

```
[8] base
```

```
other attached packages:
```

```
[1] EnhancedVolcano_1.24.0 ComplexHeatmap_2.22.0 ggsurvfit_1.2.0
```

```
[4] ggrepel_0.9.8          tidyr_1.3.2          dplyr_1.2.0
```

```
[7] pheatmap_1.0.13        survminer_0.5.2      survival_3.8-6
```

```
[10] ggpubr_0.6.3           ggplot2_4.0.2
```

```
loaded via a namespace (and not attached):
```

```
[1] shape_1.4.6.1          circlize_0.4.17       gtable_0.3.6
```

```
[4] rjson_0.2.23           xfun_0.57             GlobalOptions_0.1.3
```

```
[7] rstatix_0.7.3          lattice_0.22-9        vctrs_0.7.1
```

```
[10] tools_4.4.3            generics_0.1.4        stats4_4.4.3
```

```
[13] parallel_4.4.3         tibble_3.3.1          cluster_2.1.8.2
```

```
[16] pkgconfig_2.0.3        Matrix_1.7-5          RColorBrewer_1.1-3
```

```
[19] S4Vectors_0.44.0       S7_0.2.1              lifecycle_1.0.5
```

```
[22] compiler_4.4.3         farver_2.1.2          codetools_0.2-20
```

```
[25] clue_0.3-68            carData_3.0-6         htmltools_0.5.9
```

```
[28] yaml_2.3.12            Formula_1.2-5         crayon_1.5.3
```

```
[31] pillar_1.11.1          car_3.1-5             iterators_1.0.14
```

[34]	abind_1.4-8	foreach_1.5.2	nlme_3.1-168
[37]	tidyselect_1.2.1	digest_0.6.39	purrr_1.2.1
[40]	labeling_0.4.3	splines_4.4.3	fastmap_1.2.0
[43]	colorspace_2.1-2	cli_3.6.6	magrittr_2.0.4
[46]	dichromat_2.0-0.1	broom_1.0.12	withr_3.0.2
[49]	scales_1.4.0	backports_1.5.1	rmarkdown_2.30
[52]	matrixStats_1.5.0	otel_0.2.0	gridExtra_2.3
[55]	ggsignif_0.6.4	png_0.1-9	GetoptLong_1.1.0
[58]	evaluate_1.0.5	knitr_1.51	IRanges_2.40.1
[61]	doParallel_1.0.17	mgcv_1.9-4	rlang_1.1.7
[64]	Rcpp_1.1.1	glue_1.8.0	BiocGenerics_0.52.0
[67]	rstudioapi_0.18.0	jsonlite_2.0.0	R6_2.6.1